Course: Database System

Submitted to: Dr.Asif Sohail

Submited by:

- Samar Fatima (BITF24A034) •
  Aleena Mushtaq (BITF24A012) •
  Hajra Lodhi (BITF24A008)

Dated: 22-December-2025

Project: Hotel Management System

Database System-Project Report

1. Introduction

The Hotel Management System (HMS) is developed to simplify and automate the daily operations of a hotel by offering a centralized platform for handling reservations, guest records, staff activities, services, and inventory control. In many traditional hotel environments, manual procedures and disconnected systems lead to problems such as repeated data entry, reservation conflicts, delayed communication between departments, and limited access to real-time information. The HMS overcomes these issues by integrating all hotel operations into a single, well-organized database system.

The system is designed to support different categories of users, ensuring controlled access and smooth operational flow:

1. Administrators: Manage overall system configuration, maintain data accuracy, supervise hotel operations, and generate analytical and managerial reports.

2. System Users: Include hotel staff (such as receptionists, housekeeping, and maintenance personnel) and guests who interact with the system for bookings, service requests, and feedback submission.

Core Functional Components

Guest Information Management:

Stores detailed guest profiles, including contact information, stay history, and service records, allowing the hotel to deliver customized and efficient services.

Reservation and Booking Control:

Handles the complete booking process, including reservation creation, updates, cancellations, and check-incheck-out procedures, while maintaining accurate room availability.

Room and Operational Unit Management:

Manages room details, categories, and occupancy status, and coordinates hotel departments to ensure

smooth operational workflows.

Hotel Service Handling:

Maintains information about hotel services such as room service, laundry, and other amenities, and records their actual usage by guests during their stay.

Employee and Department Management:

Organizes staff records, assigns employees to departments, and defines operational roles to improve accountability and workforce efficiency.

Inventory Control System:

Tracks hotel inventory items, suppliers, purchasing activities, and item usage by various departments, helping to maintain optimal stock levels and reduce wastage.

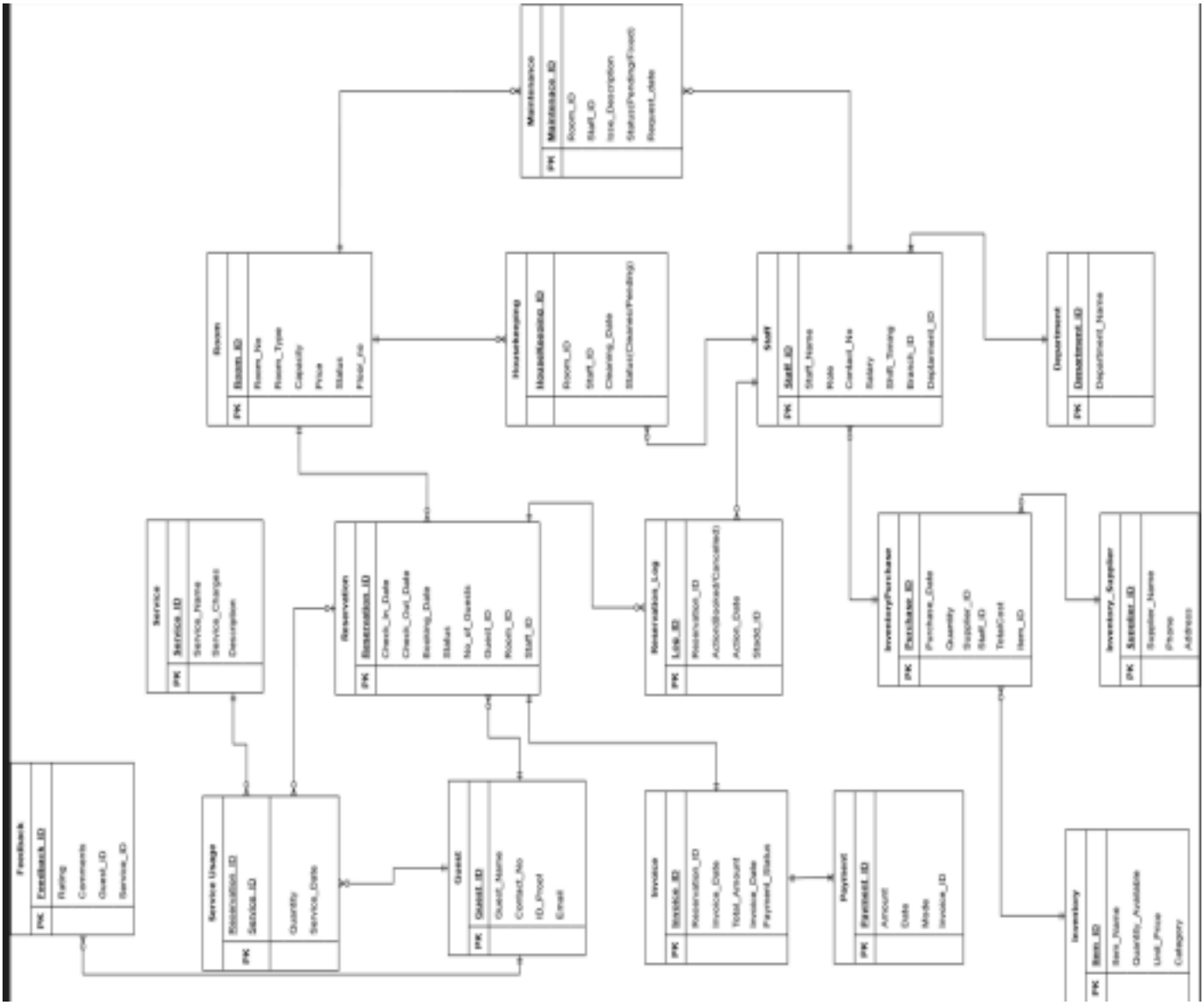
Billing and Payment Management:

Generates accurate bills for room bookings and additional services, supports multiple payment methods, and ensures secure financial transactions.

Feedback Collection and Reporting:

Collects guest reviews and feedback to support service improvement and produces reports related to occupancy rates, revenue, staff performance, and inventory utilization.

EERD



Construction of Relational schema

Bottom-Up Approach:

1. Identification of Entities and Attributes

Listing all attributes of HMS tables and their respective Primary Keys (PKs) and Foreign Keys (FKs). • GUEST Attributes (PK: GUEST_ID, GUEST_NAME, CONTACT, EMAIL, ID_PROOF. • ROOM Attributes (PK: ROOM_ID, ROOM_TYPE, PRICE, CAPACITY, STATUS, FLOOR_NO.

RESERVATION Attributes (PK: RESERVATION_ID, GUEST_ID(FK), ROOM_ID(FK), CHECKING_DATE, CHECKOUT_DATE, STATUS(Booking), STAFF_ID(FK), NO_OF_GUESTS, BOOKING_DATE. • STAFF Attributes (PK: STAFF_ID, STAFF_NAME, ROLE, SALARY, SHIFT_TIMING, DEPARTMENT_ID(FK), CONTACT_NO.

- FINANCIAL Attributes (PKs: INVOICE_ID, PAYMENT_ID), RESERVATION_ID(FK), TOTAL_AMOUNT, INVOICE_STATUS, PAYMENT_METHOD, AMOUNT, INVOICE_ID (FK).
- SERVICEUSAGE Attributes (PKs: SERVICE_ID, Composite PK: RESERVATION_ID, SERVICE_ID), SERVICE_NAME, PRICE(Service), QUANTITY, USAGE_DATE.

2. Identification of Functional Dependencies (FDs)

We establish the FDs where the determinant is typically the Primary Key.

- GUEST_ID GUEST_NAME, EMAIL
- ROOM_IDROOM_TYPE (FK), STATUS (Room), FLOOR_NO
- STAFF_IDSTAFF_NAME, ROLE, SALARY, SHIFT_TIMING, DEPARTMENT_ID
- RESERVATION_IDGUEST_ID, ROOM_ID, CHECKIN_DATE, CHECKOUT_DATE, STATUS (Booking) •

DEPT_IDDEPT_NAME

- SERVICE_IDSERVICE_NAME, PRICE (Service)
- INVENTORY_IDITEM_NAME, CATEGORY, UNIT_PRICE, QUANTITY_AVAILABLE •

RESERVATION_ID, SERVICE_IDQUANTITY, USAGE_DATE (Composite Key of SERVICE_USAGE) **3. Normalization Process (Focus on 3NF)**

GUEST Table (GUEST_ID, GUEST_NAME, CONTACT, EMAIL, ID_PROOF)

- **1NF Check:** CONTACT → multi-valued (Guest can have multiple contact).
  ID_PROOF → multi-valued (CNIC, Passport, etc.)
- **2NF Check:** Passes (Simple Primary Key GUEST_ID).
- **3NF Check:** Passes (No transitive dependency).
  - **Action:** Decompose to resolve multi-valued attributes.

Decomposed Relation	PK	Attributes	Rationale
GUEST	GUEST_ID	GUEST_NAME, EMAIL	Core guest information.

Decomposed Relation	PK	Attributes	Rationale
GUEST_CONTACT	GUEST_ID, CONTACT (Composite)	GUEST_ID [FK], CONTACT	Separates the multi-valued attribute CONTACT to achieve 1NF.
GUEST_ID_PROOF	GUEST_ID ID_PROOF (Composite)	GUEST_ID [FK], ID_PROOF	Separates the multi-valued attribute ID_PROOF to achieve 1NF.

ROOM Table (ROOM_ID, ROOM_TYPE, PRICE, CAPACITY, STATUS, FLOOR_NO)

- **1NF2NF Check:** Passes.
- **3NF Check:** Fails due to **transitive dependency**
  ROOM_TYPE → PRICE, CAPACITY
  ROOM_ID → ROOM_TYPE → PRICE, CAPACITY
  - **Action:** Decompose to eliminate the transitive dependency.

Decomposed Relation	**PK**	**Attributes**	**Rationale**

ROOM ROOM_ID ROOM_TYPE [FK],

STATUS, FLOOR_NO Physical room details.

ROOM_TYPE	ROOM_TYPE	PRICE, CAPACITY	Details common to all rooms of that type.

STAFF Table (STAFF_ID, STAFF_NAME, ROLE, SALARY, SHIFT_TIMING, DEPARTMENT_ID, CONTACT_NO)

- **1NF Check:** Fails.
  CONTACT_NO marked UNIQUE but *still logically multi-valued*
  One staff member can have multiple contact numbers.
- **2NF Check:** Passes (No composite key dependency).
- **3NF Check:** Fails if:
  ROLE → SALARY
  STAFF_ID → ROLE → SALARY
  Then transitive dependency exists.
  - **Action:** Separate CONTACT_NO for 1NF3NF and clarify SALARY.

Decomposed Relation	**PK**	**Attributes**	**Rationale**
STAFF	STAFF_ID	STAFF_NAME, ROLE, SHIFT_TIMING,	Core employee details.

Decomposed Relation	**PK**	**Attributes**	**Rationale**

		DEPARTMENT_ ID [FK]	
ROLE	ROLE_ID	SALARY	To achieve 3NF.

SERVICE_USAGE Table (USAGE_ID, GUEST_ID, RESERVATION_ID, SERVICE_ID, QUANTITY, USAGE_DATE)

- **1NF, 2NF, 3NF Check:** Passes as defined. USAGE_ID is the PK, and all other attributes are dependent on it. This table correctly links the many-to-many relationship between RESERVATION and SERVICE.
- **RESERVATION Table** (RESERVATION_ID(PK), GUEST_ID(FK), ROOM_ID (FK), CHECKIN_DATE, CHECKOUT_DATE, STATUS, STAFF_ID(FK), NO_OF_GUESTS, BOOKING_DATE)

1NF	(Atomic values, no repeating groups).	**Satisfied**
2NF	(No partial dependencies).	**Satisfied**
3NF	(No transitive dependency).	**Satisfied**

DEPARTMENT Table (DEPT_ID (PK), DEPT_NAME)

1NF2NF	(Simple PK, Atomic values).	**Satisfied**
3NF	Since there is only one non-key attribute DEPT_NAME, no transitive dependency is possible.	**Satisfied**
Conclusion	Table in **3NF**.	

SERVICE Table (SERVICE_ID (PK), SERVICE_NAME, PRICE)

1NF2NF	(Simple PK, Atomic values).	**Satisfied**
3NF	SERVICE_NAME does not determine PRICE (Price can change regardless of the name).	**Satisfied**
Conclusion	Table in **3NF**.	

INVOICE Table (INVOICE_ID (PK), RESERVATION_ID (FK), INVOICE_DATE, TOTAL_AMOUNT, INVOICE_STATUS)

1NF2NF	(Simple PK, Atomic values).	**Satisfied**

3NF	All non-key attributes DATE, AMOUNT, STATUS depend solely on INVOICE_ID.	**Satisfied**
Conclusion	Table in **3NF.**	

PAYMENT Table (PAYMENT_ID (PK), PAYMENT_METHOD, PAYMENT_DATE, AMOUNT, INVOICE_ID (FK),

1NF2NF	(Simple PK, Atomic values).	**Satisfied**
3NF	Fail (Original). It had a redundant DATE column (violating 3NF goals). The redundant DATE column was removed.	**Corrected**
Conclusion	Table in **3NF**.	

INVENTORY_PURCHASE Table (PURCHASE_ID (PK), INVENTORY_ID (FK), SUPPLIER_ID (FK), STAFF_ID (FK), QUANTITY_PURCHASED, PURCHASE_DATE)

1NF2NF	(Simple PK, Atomic values).	**Satisfied**
3NF	Non-key attributes QUANTITY, DATE depend only on the PURCHASE_ID.	**Satisfied**
Conclusion	Table in **3NF**.	

HOUSEKEEPING Table (HK_ID (PK), ROOM_ID (FK), STAFF_ID (FK), CLEANING_DATE, STATUS)

1NF2NF	(Simple PK, Atomic values).	**Satisfied**

3NF	CLEANING_DATE does not determine STATUS (or vice versa).	**Satisfied**
Conclusion	Table in **3NF**.	

MAINTANCE Table (MAINTENANCE_ID (PK), ROOM_ID (FK), STAFF_ID (FK), ISSUE_DESCRIPTION, STATUS, REQUEST_DATE)

1NF2NF3NF	All attributes depend solely on the simple Primary Key.	**Satisfied**
Conclusion	Table in **3NF**.	

FEEDBACK Table (FEEDBACK_ID (PK), GUEST_ID (FK), RATING, COMMENT, FEEDBACK_DATE)

1NF2NF3NF	All attributes depend solely on the simple Primary Key. RATING does not determine COMMENT.	**Satisfied**

Conclusion	Table in **3NF**.	

RESERVATION_LOG Table (LOG_ID (PK), RESERVATION_ID (FK), STAFF_ID (FK), ACTION, ACTION_DATE, REMARKS)

1NF2NF3NF	• Simple PK (LOG_ID) and no dependencies between non-key attributes.	**Satisfied**
Conclusion	• Table already in **3NF.**	

4. Relationship Definition

The refined relationships, using the normalized tables, are defined by Foreign Keys.

- **GUEST RESERVATION (1: N):** GUEST_ID in RESERVATION
- **ROOM RESERVATION (1: N):** ROOM_ID in RESERVATION
- **ROOM _TYPE RESERVATION (1: N):** ROOM_TYPE in RESERVATION
- **RESERVATION SERVICE_USAGE (1: N):** RESERVATION_ID in SERVICE_USAGE •

INVOICE PAYMENT (1: N): INVOICE_ID in PAYMENT

• **STAFF MAINTANENCE (1: N):** STAFF_ID in MAINTANENCE

• **STAFF HOUSEKEEPING (1: N):** STAFF_ID in HOUSEKEEPING

TOP-DOWN APPROACH

1. Requirement Analysis and Data Collection

For the successful development of the Hotel Management System (HMS), a thorough requirement analysis and data collection process is essential. This involves gathering detailed insight like including guests, hotel staff, and administrators, to understand their specific needs and challenges. Key data points include guest preferences, room operational details, reservation schedules, payment methods, inventory levels, and feedback mechanisms. Additionally, current inefficiencies in room status tracking, staff communication, and invoice processing are identified to address them effectively. By consolidating this information, the system will ensure a user_ centric design, streamlined operations, and robust data management capabilities to meet the platform's objectives.

2. Conceptual Design (ER Model Creation)

The ER Model represents the high-level entities and their relationships. It serves as the blueprint for the system, identifying how guests interact with rooms, how staff manage cleaning and maintenance, and how financial transactions are recorded.

3. Conversion of ER Model to Relational Model

The ER diagram is converted into a set of normalized tables to ensure data integrity and minimize redundancy. This involves breaking down complex entities like Guest and Staff into related tables to handle multi-valued attributes and transitive dependencies.

4. Identification of Primary and Foreign Keys

Based on the ERD and normalized relations, the following keys are established to link the system's data:

Primary Keys (PK)

GUEST: GUEST_ID

GUEST_CONTACT: Composite key (GUEST_ID, CONTACT)

ROOM: ROOM_ID

ROOM_TYPE: ROOM_TYPE

STAFF: STAFF_ID

DEPARTMENT: DEPT_ID

RESERVATION: RESERVATION_ID

INVOICE: INVOICE_ID

PAYMENT: PAYMENT_ID

SERVICE: SERVICE_ID

SERVICE_USAGE: Composite key (RESERVATION_ID, SERVICE_ID, USAGE_DATE)

SUPPLIER: SUPPLIER_ID

INVENTORY_ITEM: INVENTORY_ID

INVENTORY_PURCHASE: PURCHASE_ID

HOUSEKEEPING: HK_ID

MAINTENANCE: MAINTENANCE_ID

FEEDBACK: FEEDBACK_ID

RESERVATION_LOG: LOG_ID

Foreign Keys (FK)

ROOM:

ROOM_TYPE → ROOM_TYPE(ROOM_TYPE)

STAFF:

DEPARTMENT_ID → DEPARTMENT(DEPT_ID)

RESERVATION:

GUEST_ID → GUEST(GUEST_ID)

ROOM_ID → ROOM(ROOM_ID)

STAFF_ID → STAFF(STAFF_ID)

INVOICE:

RESERVATION_ID → RESERVATION(RESERVATION_ID)

PAYMENT:

INVOICE_ID → INVOICE(INVOICE_ID)

SERVICE_USAGE:

RESERVATION_ID → RESERVATION(RESERVATION_ID)

SERVICE_ID → SERVICE(SERVICE_ID)

INVENTORY_PURCHASE:

INVENTORY_ID → INVENTORY_ITEM(INVENTORY_ID)

SUPPLIER_ID → SUPPLIER(SUPPLIER_ID)

STAFF_ID → STAFF(STAFF_ID)

HOUSEKEEPING:

ROOM_ID → ROOM(ROOM_ID)

STAFF_ID → STAFF(STAFF_ID)

MAINTENANCE:

ROOM_ID → ROOM(ROOM_ID)

STAFF_ID → STAFF(STAFF_ID)

5. Refinement and Validation of the Schema

Refinement is performed through normalization up to the Third Normal Form (3NF). This ensures:

1NF: Elimination of multi-valued attributes like multiple contact numbers or ID proofs by creating separate tables.**2NF**: Ensuring all non-key attributes are fully dependent on the primary key, preventing partial dependencies. **3NF**: Removing transitive dependencies, such as extracting room pricing based on type into a dedicated table

Description of the Relations GUEST

Attribute	Data Type	Size	Constraints
GUEST_ID	NUMBER	4	Primary Key
GUEST_NAME	VARCHAR2	50	Not Null
CONTACT	VARCHAR2	15	Not Null
EMAIL	VARCHAR2	255	Unique
ID_proof	VARCHAR2	20	Unique

ROOM

Attribute Name	Data Type	Size	Constraints
ROOM_ID	NUMBER	4	Primary Key

ROOM_TYPE_ID	NUMBER	4	Foreign Key (References ROOM_TYPE)
STATUS	VARCHAR2	10	Default 'Available'
FLOOR_NO	NUMBER	2	—

Room_Type

Attribute Name	Data Type	Size	Constraints
ROOM_TYPE_ID	NUMBER	4	Primary Key
ROOM_TYPE_NAME	VARCHAR2	20	Not Null
PRICE	NUMBER	8,2	Not Null, CHECK (PRICE > 0)
CAPACITY	NUMBER	2	CHECK (CAPACITY > 0)

RESERVATION

Attribute	Data Type	Size	Constraints
RESERVATION_ID	NUMBER	4	Primary Key
GUEST_ID	NUMBER	4	Foreign Key References Guest(GUEST_ID)
ROOM_ID	NUMBER	4	Foreign Key References Room(ROOM_ID)
CHECKIN_DATE	DATE	-	Not Null

CHECKOUT_DATE	DATE	-	Not Null, CHECK (CHECKOUT_DATE > CHECKIN_DATE)
STATUS	VARCHAR2	15	Default 'Booked'
STAFF_ID	NUMBER	4	Foreign Key references Staff(STAFF_ID)
NO_OF_GUESTS	NUMBER	2	CHECK (NO_OF_GUESTS > 0)

BOOKING_DATE	DATE	-	NOT NULL

STAFF

Attribute	Data Type	Size	Constraints
STAFF_ID	NUMBER	4	Primary Key
STAFF_NAME	VARCHAR2	50	Not Null
ROLE	VARCHAR2	20	Not Null
SALARY	NUMBER	10,2	Not Null
SHIFT_TIMING	DATE	-	-
DEPARTMENT_ID	NUMBER	4	FK references Department(DEPARTMENT_ID)
CONTACT_NO	VARCHAR2	15	UNIQUE

DEPARTMENT

Attribute	Data Type	Size	Constraints
DEPT_ID	NUMBER	4	Primary Key
DEPT_NAME	VARCHAR2	30	Not Null

SERVICE

Attribute	Data Type	Size	Constraints
SERVICE_ID	NUMBER	4	Primary Key
SERVICE_NAME	VARCHAR2	50	Not Null
PRICE	NUMBER	8,2	CHECK (SERVICE_CHARGE > 0)

SERVICE_USAGE

Attribute	Data Type	Size	Constraints
RESERVATION_ID	NUMBER	4	PK, Foreign Key references

			Reservation(RESERVATION_ID)
SERVICE_ID	NUMBER	4	PK, Foreign Key references Service(SERVICE_ID)
QUANTITY	NUMBER	3	Not Null
USAGE_DATE	DATE	-	Not Null

INVENTORY_ITEM

Attribute	Data Type	Size	Constraints
INVENTORY_ID	NUMBER	4	Primary Key
ITEM_NAME	VARCHAR2	50	Not Null
CATEGORY	VARCHAR2	30	-
UNIT_PRICE	NUMBER	10,2	CHECK (UNIT_PRICE > 0)
QUANTITY_AVAILABLE	NUMBER	5	CHECK (QUANTITY_AVAILABLE >= 0)

SUPPLIER

Attribute	Data Type	Size	Constraints
SUPPLIER_ID	NUMBER	4	Primary Key
SUPPLIER_NAME	VARCHAR2	50	Not Null
CONTACT	VARCHAR2	20	Not Null
ADDRESS	VARCHAR2	200	-

INVENTORY_PURCHASE

Attribute	Data Type	Size	Constraints
PURCHASE_ID	NUMBER	4	Primary Key
INVENTORY_ID	NUMBER	4	Foreign Key InventoryItem(INVENTORY_ID)
SUPPLIER_ID	NUMBER	4	Foreign Key Supplier(SUPPLIER_ID)

STAFF_ID	NUMBER	4	Foreign Key references Staff(STAFF_ID)
QUANTITY_PURCHASED	NUMBER	5	Not Null
TOTAL_COST	NUMBER	10,2	CHECK (QUANTITY > 0)
PURCHASE_DATE	DATE	-	Not Null

PAYMENT

Attribute	Data Type	Size	Constraints
PAYMENT_ID	NUMBER	4	Primary Key
PAYMENT_METHOD	VARCHAR2	20	CHECK (Method IN ('Cash' ,'Card', 'Online'))
PAYMENT_DATE	DATE	-	Not Null
AMOUNT	NUMBER	10,2	CHECK (AMOUNT > 0)
INVOICE_ID			FK references Invoice(INVOICE_ID)
DATE			NOT NULL

HOUSEKEEPING

Attribute	Data Type	Size	Constraints
HOUSEKEEPING_ID	NUMBER	4	Primary key
ROOM_ID	NUMBER	4	Foreign Key references Room(ROOM_ID)
STAFF_ID	NUMBER	4	Foreign Key references Staff(STAFF_ID)
CLEANING_DATE	DATE	–	PK, Not Null
STATUS	VARCHAR	15	Not Null (Cleaned Pending)

MAINTANCE

Attribute	Data Type	Size	Constraints

MAINTENANCE_ID	NUMBER	4	Primary Key
ROOM_ID	NUMBER	4	Foreign Key references Room(ROOM_ID)
STAFF_ID	NUMBER	4	Foreign Key references Staff(STAFF_ID)
ISSUE_DESCRIPTION	VARCHAR	200	Not Null
STATUS	VARCHAR	15	CHECK (STATUS IN ('Reported', 'In-Progress', 'Resolved'))
REQUEST_DATE	DATE	–	Not Null

FEEDBACK

Attribute	Data Type	Size	Constraints
FEEDBACK_ID	NUMBER	4	Primary Key
GUEST_ID	NUMBER	4	Foreign Key references Guest(GUEST_ID)
RATING	NUMBER	1	Not Null (1–5)
COMMENT	VARCHAR	200	–
FEEDBACK_DATE	DATE	–	Not Null

INVOICE

Attribute	Data Type	Size	Constraints
INVOICE_ID	NUMBER	4	Primary Key
RESERVATION_ID	NUMBER	4	FK references Reservation(RESERVATION_ID
INVOICE_DATE	DATE	–	Not Null
TOTAL_AMOUNT	NUMBER	10,2	CHECK (TOTAL_AMOUNT

			>= 0)
INVOICE_STATUS	VARCHAR	15	Not Null (Paid Unpaid Partial)

RESERVATION_LOG

Attribute	Data Type	Description Notes	Constraints
LOG_ID	NUMBER	Unique ID for each log entry	**PK**, auto-increment
RESERVATION_ID	NUMBER	The reservation being logged	FK references Reservation(Reservation_ID), NOT NULL
STAFF_ID	NUMBER	Who made the change (guststaffadmin)	FK references STAFF(STAFF_ID),NOT NULL
ACTION	VARCHAR2(50)	Action performed (Created, Updated, Cancelled)	NOT NULL, CHECK constraint
ACTION_DATE	DATE	Timestamp of the action	Default SYSDATE, NOT NULL
PREVIOUS_STATUS	VARCHAR2(15)	Status before the change	Optional
NEW_STATUS	VARCHAR2(15)	Status after the change	Optional
Remarks	VARCHAR2(255)	Optional notes or comments	Optional

CREATE TABLE STATEMENTS:

1. Department Table

```
CREATE TABLE DEPARTMENT
 (
 DEPT_ID NUMBER (4) PRIMARY KEY,
 DEPT_NAME VARCHAR2(30) NOT NULL
);
```

Object Type **TABLE** Object **DEPARTMENT**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
DEPARTMENT	DEPT_ID	NUMBER	-	4	0	1	-	-	-
	DEPT_NAME	VARCHAR2	30	-	-	-	-	-	-
									1 - 2

2. Staff Table

```
CREATE TABLE STAFF (
 STAFF_ID NUMBER (4) PRIMARY KEY,
 STAFF_NAME VARCHAR2(50) NOT NULL,
 ROLE VARCHAR2(20) NOT NULL,
 SALARY NUMBER (10,2) CHECK (SALARY > 0),
 SHIFT_TIMING VARCHAR2(20),
 DEPT_ID NUMBER (4),
 CONSTRAINT FK_DEPT
 FOREIGN KEY (DEPT_ID)
 REFERENCES DEPARTMENT(DEPT_ID)
);
```

Object Type **TABLE** Object **STAFF**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
STAFF	STAFF_ID	NUMBER	-	4	0	1	-	-	-
	STAFF_NAME	VARCHAR2	50	-	-	-	-	-	-
	ROLE	VARCHAR2	20	-	-	-	-	-	-
	SALARY	NUMBER	-	10	2	-	✓	-	-
	SHIFT_TIMING	VARCHAR2	20	-	-	-	✓	-	-
	DEPT_ID	NUMBER	-	4	0	-	✓	-	-
									1 - 6

3. Room Type Table

```
CREATE TABLE ROOM_TYPE (
 ROOM_TYPE VARCHAR2(20) PRIMARY KEY,
 PRICE NUMBER (8,2) NOT NULL CHECK (PRICE > 0),
 CAPACITY NUMBER (2) NOT NULL CHECK (CAPACITY > 0)
);
```

Object Type **TABLE** Object **ROOM_TYPE**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
ROOM_TYPE	ROOM_TYPE	VARCHAR2	20	-	-	1	-	-	-
	PRICE	NUMBER	-	8	2	-	-	-	-
	CAPACITY	NUMBER	-	2	0	-	-	-	-
									1 - 3

4. Room Table

```
CREATE TABLE ROOM (
ROOM_ID NUMBER (4) PRIMARY KEY,
ROOM_TYPE VARCHAR2(20) REFERENCES ROOM_TYPE(ROOM_TYPE),
STATUS VARCHAR2(10) DEFAULT 'Available',
FLOOR_NO NUMBER (2) NOT NULL
);
```

Object Type **TABLE** Object **ROOM**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
ROOM	ROOM_ID	NUMBER	-	4	0	1	-	-	-
	ROOM_TYPE	VARCHAR2	20	-	-	-	✓	-	-
	STATUS	VARCHAR2	10	-	-	-	✓	'Available'	-
	FLOOR_NO	NUMBER	-	2	0	-	-	-	-
									1 - 4

5. Guest Table

```
CREATE TABLE GUEST (
GUEST_ID NUMBER (4) PRIMARY KEY,
GUEST_NAME VARCHAR2(50) NOT NULL,
EMAIL VARCHAR2(255) UNIQUE NOT NULL
);
```

Object Type **TABLE** Object **GUEST**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
GUEST	GUEST_ID	NUMBER	-	4	0	1	-	-	-
	GUEST_NAME	VARCHAR2	50	-	-	-	-	-	-
	EMAIL	VARCHAR2	255	-	-	-	-	-	-
									1 - 3

6. Guest Contact Table

```
CREATE TABLE GUEST_CONTACT (
```

GUEST_ID NUMBER (4) REFERENCES GUEST(GUEST_ID),

CONTACT VARCHAR2(15),

PRIMARY KEY (GUEST_ID, CONTACT)

);

Object Type **TABLE** Object **GUEST_CONTACT**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
GUEST_CONTACT	GUEST_ID	NUMBER	-	4	0	1	-	-	-
	CONTACT	VARCHAR2	15	-	-	2	-	-	-
									1 - 2

7.SERVICE TABLE

CREATE TABLE SERVICE (

SERVICE_ID NUMBER (4) PRIMARY KEY,

SERVICE_NAME VARCHAR2(30) NOT NULL,

PRICE NUMBER (8,2) NOT NULL

);

Object Type **TABLE** Object **SERVICE**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
SERVICE	SERVICE_ID	NUMBER	-	4	0	1	-	-	-
	SERVICE_NAME	VARCHAR2	30	-	-	-	-	-	-
	PRICE	NUMBER	-	8	2	-	-	-	-
									1 - 3

8. Reservation Table

CREATE TABLE RESERVATION (

RESERVATION_ID NUMBER (4) PRIMARY KEY,

GUEST_ID NUMBER (4) REFERENCES GUEST(GUEST_ID),
ROOM_ID NUMBER (4) REFERENCES ROOM(ROOM_ID),

STAFF_ID NUMBER (4) REFERENCES STAFF(STAFF_ID),

CHECKIN_DATE DATE NOT NULL,

CHECKOUT_DATE DATE NOT NULL,

STATUS VARCHAR2(15) DEFAULT 'Booked',

NO_OF_GUESTS NUMBER (2) CHECK (NO_OF_GUESTS > 0),

BOOKING_DATE DATE DEFAULT SYSDATE,

CONSTRAINT CHK_DATES CHECK (CHECKOUT_DATE > CHECKIN_DATE)

);

Object Type **TABLE** Object **RESERVATION**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
RESERVATION	RESERVATION_ID	NUMBER	-	4	0	1	-	-	-
	GUEST_ID	NUMBER	-	4	0	-	✓	-	-
	ROOM_ID	NUMBER	-	4	0	-	✓	-	-
	STAFF_ID	NUMBER	-	4	0	-	✓	-	-
	CHECKIN_DATE	DATE	7	-	-	-	-	-	-
	CHECKOUT_DATE	DATE	7	-	-	-	-	-	-
	STATUS	VARCHAR2	15	-	-	-	✓	'Booked'	-
	NO_OF_GUESTS	NUMBER	-	2	0	-	✓	-	-
	BOOKING_DATE	DATE	7	-	-	-	✓	SYSDATE	-
									1 - 9

9. Service Usage Table (Associative Table for M:N Relationship)

CREATE TABLE SERVICE_USAGE (

RESERVATION_ID NUMBER (4) REFERENCES RESERVATION(RESERVATION_ID),

SERVICE_ID NUMBER (4) REFERENCES SERVICE(SERVICE_ID),

QUANTITY NUMBER (3) DEFAULT 1 CHECK (QUANTITY > 0),

USAGE_DATE DATE DEFAULT SYSDATE,

PRIMARY KEY (RESERVATION_ID, SERVICE_ID, USAGE_DATE)

);

Object Type **TABLE** Object **SERVICE_USAGE**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
SERVICE_USAGE	RESERVATION_ID	NUMBER	-	4	0	1	-	-	-
	SERVICE_ID	NUMBER	-	4	0	2	-	-	-
	QUANTITY	NUMBER	-	3	0	-	✓	1	-
	USAGE_DATE	DATE	7	-	-	3	-	SYSDATE	-
									1 - 4

10. Invoice Table

CREATE TABLE INVOICE (

INVOICE_ID NUMBER (4) PRIMARY KEY,

RESERVATION_ID NUMBER (4) NOT NULL,

INVOICE_DATE DATE NOT NULL,

TOTAL_AMOUNT NUMBER (10,2) DEFAULT 0 CHECK (TOTAL_AMOUNT >= 0),

INVOICE_STATUS VARCHAR2(15) NOT NULL,

CONSTRAINT FK_RESERVATION FOREIGN KEY (RESERVATION_ID) REFERENCES

RESERVATION(RESERVATION_ID)

);

Object Type **TABLE** Object **INVOICE**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
INVOICE	INVOICE_ID	NUMBER	-	4	0	1	-	-	-
	RESERVATION_ID	NUMBER	-	4	0	-	-	-	-
	INVOICE_DATE	DATE	7	-	-	-	-	-	-
	TOTAL_AMOUNT	NUMBER	-	10	2	-	✓	0	-
	INVOICE_STATUS	VARCHAR2	15	-	-	-	-	-	-
									1 - 5

11. Payment Table

CREATE TABLE PAYMENT (

PAYMENT_ID NUMBER (4) PRIMARY KEY,

INVOICE_ID NUMBER (4) REFERENCES INVOICE(INVOICE_ID),

PAYMENT_METHOD VARCHAR2(20) NOT NULL,

PAYMENT_DATE DATE DEFAULT SYSDATE,

AMOUNT NUMBER (10,2) NOT NULL CHECK (AMOUNT > 0)

);

Object Type **TABLE** Object **PAYMENT**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
PAYMENT	PAYMENT_ID	NUMBER	-	4	0	1	-	-	-
	INVOICE_ID	NUMBER	-	4	0	-	✓	-	-
	PAYMENT_METHOD	VARCHAR2	20	-	-	-	-	-	-
	PAYMENT_DATE	DATE	7	-	-	-	✓	SYSDATE	-
	AMOUNT	NUMBER	-	10	2	-	-	-	-
									1 - 5

12.Supplier Table

CREATE TABLE SUPPLIER (

SUPPLIER_ID NUMBER(4) PRIMARY KEY,

SUPPLIER_NAME VARCHAR2(50) NOT NULL,

CONTACT VARCHAR2(15),

ADDRESS VARCHAR2(100)

);

Object Type **TABLE** Object **SUPPLIER**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
SUPPLIER	SUPPLIER_ID	NUMBER	-	4	0	1	-	-	-
	SUPPLIER_NAME	VARCHAR2	50	-	-	-	-	-	-
	CONTACT	VARCHAR2	15	-	-	-	✓	-	-
	ADDRESS	VARCHAR2	100	-	-	-	✓	-	-
									1 - 4

13.Inventory Item Table

```
CREATE TABLE INVENTORY_ITEM (
 INVENTORY_ID NUMBER(4) PRIMARY KEY,
 ITEM_NAME VARCHAR2(50) NOT NULL,
 CATEGORY VARCHAR2(30),
 UNIT_PRICE NUMBER(8,2) CHECK (UNIT_PRICE >= 0),
 STOCK_QTY NUMBER(5) DEFAULT 0
);
```

Object Type **TABLE** Object **INVENTORY_ITEM**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
INVENTORY_ITEM	INVENTORY_ID	NUMBER	-	4	0	1	-	-	-
	ITEM_NAME	VARCHAR2	50	-	-	-	-	-	-
	CATEGORY	VARCHAR2	30	-	-	-	✓	-	-
	UNIT_PRICE	NUMBER	-	8	2	-	✓	-	-
	STOCK_QTY	NUMBER	-	5	0	-	✓	0	-
									1 - 5

14.Inventory Purchase Table

```
CREATE TABLE INVENTORY_PURCHASE (
 PURCHASE_ID NUMBER (4) PRIMARY KEY,
 INVENTORY_ID NUMBER (4) REFERENCES INVENTORY_ITEM(INVENTORY_ID),
 SUPPLIER_ID NUMBER (4) REFERENCES SUPPLIER(SUPPLIER_ID),
 STAFF_ID NUMBER (4) REFERENCES STAFF(STAFF_ID),
 QTY_PURCHASED NUMBER (5) NOT NULL CHECK (QTY_PURCHASED > 0),
 TOTAL_COST NUMBER (10,2),
 PURCHASE_DATE DATE DEFAULT SYSDATE
);
```

Object Type **TABLE** Object **INVENTORY_PURCHASE**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
INVENTORY_PURCHASE	PURCHASE_ID	NUMBER	-	4	0	1	-	-	-
	INVENTORY_ID	NUMBER	-	4	0	-	✓	-	-
	SUPPLIER_ID	NUMBER	-	4	0	-	✓	-	-
	STAFF_ID	NUMBER	-	4	0	-	✓	-	-
	QTY_PURCHASED	NUMBER	-	5	0	-	-	-	-
	TOTAL_COST	NUMBER	-	10	2	-	✓	-	-
	PURCHASE_DATE	DATE	7	-	-	-	✓	SYSDATE	-
									1 - 7

15. Reservation Log Table

CREATE TABLE RESERVATION_LOG (

LOG_ID NUMBER (6) PRIMARY KEY,

RESERVATION_ID NUMBER (4) REFERENCES RESERVATION(RESERVATION_ID),

ACTION VARCHAR2(50) NOT NULL,

ACTION_DATE DATE DEFAULT SYSDATE,

REMARKS VARCHAR2(255)

);

16.HOUSEKEEPING TABLE

CREATE TABLE HOUSEKEEPING (

HK_ID NUMBER(4) PRIMARY KEY,

ROOM_ID NUMBER(4) REFERENCES ROOM(ROOM_ID),
STAFF_ID NUMBER(4) REFERENCES STAFF(STAFF_ID),

CLEANING_DATE DATE DEFAULT SYSDATE,

STATUS VARCHAR2(15) CHECK (STATUS IN ('Clean', 'Dirty', 'In-Progress'))

);

17. Maintenance Table

```
CREATE TABLE MAINTENANCE (
 MAINTENANCE_ID NUMBER (4) PRIMARY KEY,
 ROOM_ID NUMBER (4) REFERENCES ROOM(ROOM_ID),
 STAFF_ID NUMBER (4) REFERENCES STAFF(STAFF_ID),
 DESCRIPTION VARCHAR2(200) NOT NULL,
 STATUS VARCHAR2(15) CHECK (STATUS IN ('Reported', 'In-Progress', 'Resolved')),
REQUEST_DATE DATE DEFAULT SYSDATE
);
```

VIEWS:

1. Shows the Guest names along with their Room IDs and Check-inCheck-out dates.

```
CREATE OR REPLACE VIEW VIEW_GUEST_STAY_DETAILS AS
SELECT G. GUEST_NAME, R. ROOM_ID, R. CHECKIN_DATE, R. CHECKOUT_DATE,
 R. STATUS
FROM GUEST G
JOIN RESERVATION R ON G. GUEST_ID = R. GUEST_ID;
```

2.

Shows each Room ID with its Category (Type), Price per night, and current Availability.

```
CREATE OR REPLACE VIEW VIEW_ROOM_PRICING AS
SELECT RM. ROOM_ID, RT. ROOM_TYPE, RT. PRICE, RT. CAPACITY, RM. STATUS
FROM ROOM RM
JOIN ROOM_TYPE RT ON RM. ROOM_TYPE = RT. ROOM_TYPE;
```

3.Shows Invoice IDs and Reservation details for guests who have not cleared their full payments.

```
CREATE OR REPLACE VIEW VIEW_PENDING_PAYMENTS AS
SELECT I. INVOICE_ID, I. RESERVATION_ID, I. TOTAL_AMOUNT, I. INVOICE_STATUS
FROM INVOICE I
WHERE I. INVOICE_STATUS IN ('Unpaid', 'Partial');
```

4. Shows Staff names along with the total count of Reservations they have processed.

```
CREATE OR REPLACE VIEW VIEW_STAFF_WORKLOAD AS
SELECT S. STAFF_NAME, S. ROLE, COUNT (R. RESERVATION_ID) AS TOTAL_BOOKINGS
FROM STAFF S
LEFT JOIN RESERVATION R ON S. STAFF_ID = R. STAFF_ID
GROUP BY S. STAFF_NAME, S. ROLE;
```

5. Shows the total Amount collected per day along with the Payment Method used.

```
CREATE OR REPLACE VIEW VIEW_DAILY_FINANCE AS

SELECT PAYMENT_DATE, PAYMENT_METHOD, SUM(AMOUNT) AS DAILY_TOTAL

FROM PAYMENT

GROUP BY PAYMENT_DATE, PAYMENT_METHOD;
```

6.Shows the Inventory Purchase details along with Supplier info, Item names, and the Staff member who managed the purchase.

```
CREATE OR REPLACE VIEW VIEW_PURCHASE_HISTORY AS

SELECT

 P. PURCHASE_ID,

 I.ITEM_NAME,

 S. SUPPLIER_NAME,

 P.QTY_PURCHASED,
 P. TOTAL_COST,

 ST. STAFF_NAME AS MANAGED_BY,

 P. PURCHASE_DATE

FROM INVENTORY_PURCHASE P

JOIN INVENTORY_ITEM I ON P. INVENTORY_ID = I. INVENTORY_ID

JOIN SUPPLIER S ON P. SUPPLIER_ID = S. SUPPLIER_ID

JOIN STAFF ST ON P. STAFF_ID = ST. STAFF_ID;
```

7.Shows a complete history of Actions (Created, Cancelled, etc.) taken on each Reservation ID.

```
CREATE OR REPLACE VIEW VIEW_RESERVATION_HISTORY AS
SELECT RL.LOG_ID, RL. RESERVATION_ID, RL. ACTION, RL. ACTION_DATE, RL. REMARKS
FROM RESERVATION_LOG RL;
```

8. Shows the Cleaning Status of each Room along with the assigned Housekeeping Staff.

```
CREATE OR REPLACE VIEW VIEW_CLEANING_SCHEDULE AS
SELECT H.HK_ID, H. ROOM_ID, S. STAFF_NAME AS CLEANER, H. STATUS, H. CLEANING_DATE
FROM HOUSEKEEPING H
JOIN STAFF S ON H. STAFF_ID = S. STAFF_ID;
```

9.Shows the Guest's Service usage along with Service Names and Total Cost per service.

```
CREATE OR REPLACE VIEW VIEW_SERVICE_CONSUMPTION AS

SELECT
 R. RESERVATION_ID,
 G. GUEST_NAME,
 S. SERVICE_NAME,
 SU. QUANTITY,
 (S. PRICE * SU. QUANTITY) AS TOTAL_SERVICE_COST
```

```
FROM SERVICE_USAGE SU

JOIN SERVICE S ON SU. SERVICE _ID = S. SERVICE_ID

JOIN RESERVATION R ON SU. RESERVATION _ID = R. RESERVATION_ID

JOIN GUEST G ON R. GUEST_ID = G. GUEST_ID;
```

10.Shows the Room Maintenance issues along with the assigned Technician and the current Status.

```
CREATE OR REPLACE VIEW VIEW_ACTIVE_MAINTENANCE AS

SELECT M. MAINTENANCE_ID, M. ROOM_ID, S. STAFF_NAME AS TECHNICIAN, M. DESCRIPTION, M. STATUS

FROM MAINTENANCE M

JOIN STAFF S ON M. STAFF_ID = S. STAFF_ID

WHERE M. STATUS! = 'Resolved'
```

Select Statements

1. COMPLETE RESERVATION PROFILE WITH GUEST, ROOM AND STAFF

```
SELECT

 R.RESERVATION_ID,

 G.GUEST_NAME,

 RM.ROOM_ID,

 RM.ROOM_TYPE,

 S.STAFF_NAME,

 R.CHECKIN_DATE,
```

```
 R.CHECKOUT_DATE,
 R.STATUS
FROM
 RESERVATION R
JOIN GUEST G ON R.GUEST_ID = G.GUEST_ID
JOIN ROOM RM ON R.ROOM_ID = RM.ROOM_ID
JOIN STAFF S ON R.STAFF_ID = S.STAFF_ID;
```

2. CURRENTLY OCCUPIED ROOMS WITH STAY DURATION

```
SELECT
 RM.ROOM_ID,
 RM.ROOM_TYPE,
 G.GUEST_NAME,
 (R.CHECKOUT_DATE - R.CHECKIN_DATE) AS STAY_DAYS
FROM
 ROOM RM
JOIN RESERVATION R ON RM.ROOM_ID = R.ROOM_ID
JOIN GUEST G ON R.GUEST_ID = G.GUEST_ID
WHERE
 RM.STATUS = 'Occupied';
```

3. TOTAL SERVICE CHARGES PER RESERVATION

```
SELECT
 R.RESERVATION_ID,
 SUM(SU.QUANTITY * S.PRICE) AS TOTAL_SERVICE_COST
FROM
```

```
 RESERVATION R
JOIN SERVICE_USAGE SU ON R.RESERVATION_ID = SU.RESERVATION_ID
JOIN SERVICE S ON SU.SERVICE_ID = S.SERVICE_ID GROUP BY
 R.RESERVATION_ID;
```

4. FINAL BILL SUMMARY PER GUEST

```
SELECT
 G.GUEST_NAME,
 I.INVOICE_ID,
 I.TOTAL_AMOUNT,
 I.INVOICE_STATUS
FROM
 INVOICE I
JOIN RESERVATION R ON I.RESERVATION_ID = R.RESERVATION_ID
JOIN GUEST G ON R.GUEST_ID = G.GUEST_ID;
```

5. IDENTIFY INVOICES WHERE PAYMENTS ARE STILL PENDING

```
SELECT
 I.INVOICE_ID,
 I.RESERVATION_ID,
 I.TOTAL_AMOUNT
FROM
 INVOICE I
```

```
WHERE
 I.TOTAL_AMOUNT >
 (
 SELECT NVL(SUM(P.AMOUNT), 0)
 FROM PAYMENT P
 WHERE P.INVOICE_ID = I.INVOICE_ID
 );
```

6. MOST FREQUENTLY USED SERVICES (TOP SERVICES REPORT)

```
SELECT

 S. SERVICE_NAME,
 SUM (SU. QUANTITY) AS TOTAL_USAGE
FROM
 SERVICE_USAGE SU
JOIN SERVICE S ON SU. SERVICE_ID = S.SERVICE_ID
GROUP BY
 S. SERVICE_NAME
ORDER BY
 TOTAL_USAGE DESC;
```

7. ROOMS CURRENTLY UNDER MAINTENANCE WITH STAFF DETAILS

```
SELECT
 RM. ROOM_ID,
 S. STAFF_NAME,
 M.DESCRIPTION,
 M.STATUS
FROM
 MAINTENANCE M
JOIN ROOM RM ON M.ROOM_ID = RM.ROOM_ID
JOIN STAFF S ON M.STAFF_ID = S.STAFF_ID
WHERE
 M.STATUS <> 'Resolved';
```

8. INVENTORY ITEMS THAT REQUIRE REORDERING

```
SELECT
 INVENTORY_ID,
 ITEM_NAME,
 STOCK_QTY
FROM
 INVENTORY_ITEM
WHERE
```

```
STOCK_QTY <
(SELECT AVG(STOCK_QTY) FROM INVENTORY_ITEM);
```

9. DEPARTMENT-WISE STAFF COUNT AND AVERAGE SALARY

```
SELECT
 D.DEPT_NAME,
 COUNT(S.STAFF_ID) AS TOTAL_STAFF,
 AVG(S.SALARY) AS AVERAGE_SALARY
FROM
 DEPARTMENT D
JOIN STAFF S ON D.DEPT_ID = S.DEPT_ID
GROUP BY
 D.DEPT_NAME;
```

10. ROOMS THAT HAVE NOT BEEN CLEANED IN LAST 2 DAYS

```
SELECT
 RM.ROOM_ID,
 RM.ROOM_TYPE,
 H.STATUS,
 H.CLEANING_DATE
FROM
 ROOM RM
```

```
JOIN HOUSEKEEPING H ON RM.ROOM_ID = H.ROOM_ID
WHERE
 H.CLEANING_DATE < SYSDATE - 2
 AND H.STATUS != 'Clean';
```

Functions, Procedure, Trigger

1. AUTOMATIC PRICE UPDATER

```
CREATE OR REPLACE PROCEDURE UPDATE_PRICE
(
 P_ROOM_TYPE IN VARCHAR2,
 P_NEW_PRICE IN NUMBER
)
AS
BEGIN
 UPDATE ROOM_TYPE
 SET PRICE = P_NEW_PRICE
 WHERE ROOM_TYPE = P_ROOM_TYPE;
 DBMS_OUTPUT.PUT_LINE('PRICE UPDATED FOR ROOM TYPE: ' || P_ROOM_TYPE);
END;


BEGIN
 UPDATE_PRICE('Deluxe', 15000);
END;
```

2.ROOM STATUS CHECKER

```
CREATE OR REPLACE PROCEDURE CHECK_ROOM_STATUS
(
 P_ROOM_ID IN NUMBER
)
AS
 V_STATUS ROOM.STATUS%TYPE;
BEGIN
 SELECT STATUS
 INTO V_STATUS
 FROM ROOM
 WHERE ROOM_ID = P_ROOM_ID;
 DBMS_OUTPUT.PUT_LINE('ROOM STATUS IS: ' || V_STATUS);
END;

BEGIN
 CHECK_ROOM_STATUS(201); END;
```

3.TOTAL BILL CALCULATOR (Function)

```
CREATE OR REPLACE FUNCTION GET_TOTAL_BILL(RES_ID IN NUMBER) RETURN NUMBER IS
```

```
V_TOTAL NUMBER;
BEGIN
 SELECT TOTAL_AMOUNT INTO V_TOTAL FROM INVOICE WHERE RESERVATION_ID = RES_ID;
RETURN V_TOTAL;
END;

SELECT GET_TOTAL_BILL(401) FROM DUAL;
```

4.Room Status Update

```
CREATE OR REPLACE TRIGGER TRG_ROOM_BOOKED
AFTER INSERT ON RESERVATION
FOR EACH ROW
BEGIN
 UPDATE ROOM
 SET STATUS = 'Occupied'
 WHERE ROOM_ID = :NEW.ROOM_ID;
END;
```

5. STOP BOOKING IF ROOM IS NOT AVAILABLE (Trigger)

```
CREATE OR REPLACE TRIGGER TRG_STOP_BOOKING
BEFORE INSERT ON RESERVATION
FOR EACH ROW
DECLARE
 V_STATUS VARCHAR2(20);
```

```
BEGIN
 SELECT STATUS INTO V_STATUS FROM ROOM WHERE ROOM_ID = :NEW.ROOM_ID;  IF
V_STATUS != 'Available' THEN
 RAISE_APPLICATION_ERROR(-20001, 'SORRY! THIS ROOM IS NOT EMPTY.');
END IF;
END;
INSERT INTO RESERVATION
(RESERVATION_ID, GUEST_ID, ROOM_ID, STAFF_ID,
 CHECKIN_DATE, CHECKOUT_DATE, NO_OF_GUESTS)
VALUES
(403, 2, 202, 101,
 TO_DATE('2025-02-05','YYYY-MM-DD'),
 TO_DATE('2025-02-06','YYYY-MM-DD'),
 1);
```

6. AUTOMATIC MAINTENANCE LOG (Trigger)

```
CREATE OR REPLACE TRIGGER TRG_MAINTENANCE_DONE
AFTER UPDATE OF STATUS ON MAINTENANCE
FOR EACH ROW
WHEN (NEW.STATUS = 'Finished')
BEGIN
 UPDATE ROOM SET STATUS = 'Available' WHERE ROOM_ID = :NEW.ROOM_ID;
END;
```

7. STAFF SALARY COUNTER (Function)

```
CREATE OR REPLACE FUNCTION DEPT_SALARY(D_ID IN NUMBER) RETURN NUMBER IS
V_SUM NUMBER;
BEGIN
 SELECT SUM(SALARY) INTO V_SUM FROM STAFF WHERE DEPT_ID = D_ID;
RETURN V_SUM;
END;
SELECT DEPT_SALARY(10) FROM DUAL;
```

8.TOTAL REVENUE PER ROOM TYPE (FUNCTION)

Calculates exactly how much money a specific room category (e.g., 'Deluxe') has generated.

```
CREATE OR REPLACE FUNCTION GET_ROOM_TYPE_REVENUE
(
 P_ROOM_TYPE IN VARCHAR2
)
RETURN NUMBER
IS
 V_TOTAL_REV NUMBER;
BEGIN
 SELECT SUM(I.TOTAL_AMOUNT)
 INTO V_TOTAL_REV
 FROM INVOICE I
 JOIN RESERVATION R ON I.RESERVATION_ID = R.RESERVATION_ID
 JOIN ROOM RM ON R.ROOM_ID = RM.ROOM_ID
 WHERE RM.ROOM_TYPE = P_ROOM_TYPE;

 RETURN NVL(V_TOTAL_REV, 0);
END;
```

```
SELECT GET_ROOM_TYPE_REVENUE('Deluxe') FROM DUAL;
```

9.MONTHLY TAX CALCULATOR (PROCEDURE)

A professional procedure that calculates a 10% tax on all invoices for the current month and outputs the total tax liability.

```
CREATE OR REPLACE PROCEDURE CALCULATE_MONTHLY_TAX
AS
 V_TAX_TOTAL NUMBER;
BEGIN
 SELECT SUM(TOTAL_AMOUNT * 0.10)
 INTO V_TAX_TOTAL
 FROM INVOICE
 WHERE EXTRACT(MONTH FROM INVOICE_DATE) = EXTRACT(MONTH FROM SYSDATE)
AND EXTRACT(YEAR FROM INVOICE_DATE) = EXTRACT(YEAR FROM SYSDATE);

 DBMS_OUTPUT.PUT_LINE(
 'TOTAL TAX DUE FOR CURRENT MONTH: ' || NVL(V_TAX_TOTAL, 0)
 );
END;
BEGIN
 CALCULATE_MONTHLY_TAX;
END;
```